SOLARINET · INTEGRATION REFERENCE · 2026-10-21

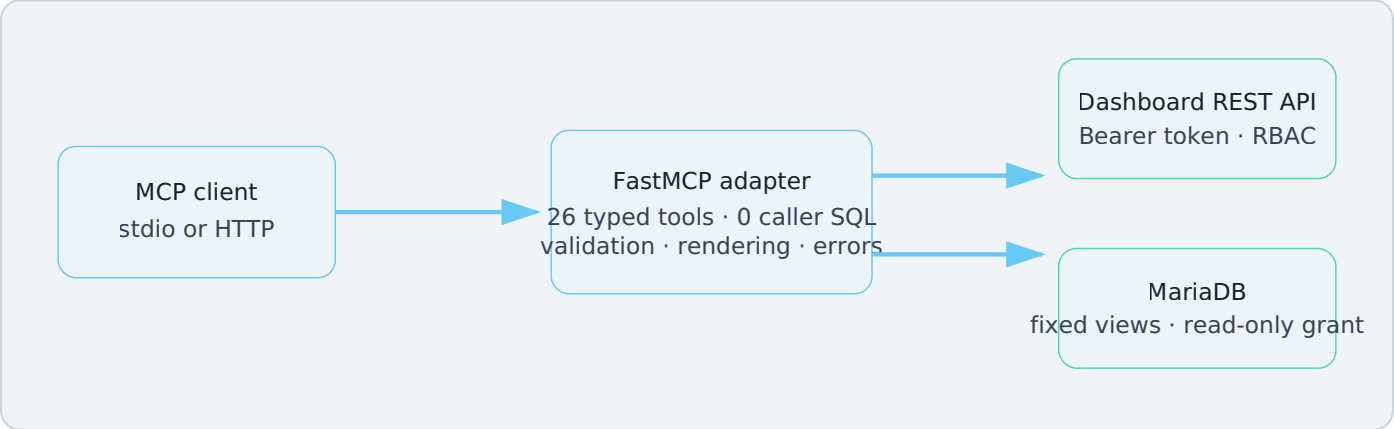
MCP Developer Reference

Implementation guide for the Python/FastMCP adapter in `integrations/mcp`. The server exposes the SolariNet REST contract and three fixed database views to MCP clients.

---

1. Component boundary

The MCP server is an adapter. It translates MCP tool inputs into calls to the dashboard API or fixed read-only database operations, then converts results into JSON or compact Markdown. The dashboard API remains the authorization boundary for REST operations; the database account should independently be read-only.



> **Current scope:** 15 read-only REST tools, 8 REST control tools, and 3 fixed database tools. Arbitrary SQL is not exposed to MCP callers.

2. Source map

File	Responsibility
`solarinet_mcp/server.py`	FastMCP registration, typed tool inputs, REST/DB dispatch, response formatting, and transport startup.
`solarinet_mcp/config.py`	Environment-only configuration. Parses REST, database, transport, and listener-port settings.
`solarinet_mcp/rest.py`	Async HTTPX client; bearer auth, TLS verification, timeout, JSON envelope validation, and normalized API errors.

File	Responsibility
`solarinet_mcp/db.py`	PyMySQL access via `asyncio.to_thread`, fixed query registry, parameter binding, and row limits.
`solarinet_mcp/formatting.py`	JSON and Markdown rendering helpers shared by the tools.
`solarinet_mcp/__main__.py`	CLI entry point and non-starting `--check` configuration/tool listing.
`tests/`	Unit tests using mocked HTTP, injected DB runners, configuration cases, and tool behavior.
`evals/`	Ten fixture-backed questions checked by calling the actual tool functions.

3. Tool contracts

Read-only REST (15)

Tool	API operation
`solarinet_list_nodes`	`GET /api/nodes`
`solarinet_get_node`	`GET /api/nodes/{id}`
`solarinet_get_node_history`	`GET /api/nodes/{id}/history`
`solarinet_list_probes`	`GET /api/probes`
`solarinet_get_probe_history`	`GET /api/probes/{targetId}/history`
`solarinet_list_alerts`	`GET /api/alerts`
`solarinet_get_topology`	`GET /api/topology`
`solarinet_server_status`	`GET /api/server/status`
`solarinet_list_discovery`	`GET /api/discovery`
`solarinet_list_enrollments`	`GET /api/enrollments`
`solarinet_list_builds`	`GET /api/builds`
`solarinet_list_segments`	`GET /api/segments`
`solarinet_list_netgear`	`GET /api/netgear`
`solarinet_get_topology_view`	`GET /api/topology?view=monitoring|network`

Tool	API operation
`solarinet_get_config`	`GET /api/config`

Control REST (8)

Control tools call POST operations and are side-effecting. They use the configured bearer token; API role checks remain authoritative. Enrollment approval and decommission additionally require `confirm=true`. Decommission can remove the requested node scopes and retire the node, so treat it as destructive.

Tool	Operation
`solarinet_trigger_survey`	Start a survey for the requested scope.
`solarinet_push_config`	Stage a config overlay for a node.
`solarinet_adopt_discovery`	Adopt a discovered device through the dashboard API.
`solarinet_ignore_discovery`	Mark a discovery candidate ignored.
`solarinet_reject_enrollment`	Reject a pending enrollment.
`solarinet_provision`	Request provisioning for a node.
`solarinet_approve_enrollment`	Approve an enrollment and sign its CSR; requires explicit confirmation.
`solarinet_decommission`	Retire a node and remove requested data scopes; requires explicit confirmation.

Fixed database views (3)

Tool	Data returned
`solarinet_db_node_summary`	Node roster summary.
`solarinet_db_active_alerts`	Active alerts joined to their rules.
`solarinet_db_host_history`	Host history for one node, bounded by the requested row limit.

4. Configuration and startup

Variable	Default	Notes
`SOLARINET_API_BASE`	unset	Dashboard origin without trailing `/api`.
`SOLARINET_API_TOKEN`	unset	Bearer credential; required for control operations.

Variable	Default	Notes
`SOLARINET_API_TIMEOUT`	30 seconds	Per-request timeout.
`SOLARINET_API_VERIFY_TLS`	true	Set false only for controlled lab testing.
`SOLARINET_DB_HOST`	127.0.0.1	MariaDB host.
`SOLARINET_DB_PORT`	3306	MariaDB port.
`SOLARINET_DB_NAME`	solarinet	Database name.
`SOLARINET_DB_USER` / `SOLARINET_DB_PASS`	unset	Use a dedicated SELECT-only MariaDB account.
`SOLARINET_DB_TIMEOUT`	10 seconds	Connection/read timeout.
`SOLARINET_MCP_TRANSPORT`	stdio	`stdio` or `streamable_http`.
`SOLARINET_MCP_PORT`	8765	HTTP port, integer 1–65535. Invalid values fail configuration.

Start with `python -m solarinet_mcp`. Default stdio is suitable for a locally launched MCP client. To start streamable HTTP, set `SOLARINET_MCP_TRANSPORT=streamable_http`; the adapter maps this project value to the FastMCP 1.x transport name `streamable-http` and applies `SOLARINET_MCP_PORT` to the SDK listener settings.

`python -m solarinet_mcp --check` prints configuration status and registered tool names without opening a listener or connecting to the API/database. It does not print credential values.

5. Security and trust boundaries

- **HTTP exposure:** the tested FastMCP 1.30.0 runtime defaults to `127.0.0.1`; this project does not implement MCP-client authentication. Keep HTTP local or put remote access behind an authenticated, access-controlled tunnel or proxy. Do not expose the listener directly to an untrusted network.
- **REST writes:** MCP annotations help clients distinguish read-only and side-effecting tools, but annotations are not authorization. The dashboard API's bearer token and RBAC checks enforce access.
- **Database reads:** no MCP tool or DB client method accepts caller-supplied SQL. The client selects from a closed registry of reviewed statements, validates parameter count and node identifiers, binds values, and caps rows. Use a MariaDB account granted only the needed SELECT privileges.
- **Transport security:** REST TLS verification defaults on. Disabling it weakens server identity checks and is intended only for controlled testing.

- **Secret handling:** supply tokens and database passwords through the process environment/ secret manager, never source files or command-line arguments. Do not include real values in diagnostic output.

6. Errors, bounds, and execution model

The REST client requires the dashboard's JSON envelope with a `data` member, maps API error envelopes and HTTP 401/403/404 responses to concise tool errors, and reports timeout or transport failures explicitly. The database driver is synchronous, so queries run in a worker thread to avoid blocking the async tool loop.

Database reads use a closed registry of three reviewed statements. Callers select a query identifier and supply only validated values; SQL text, table names, and other identifiers cannot be submitted. Each tool applies its own row limit, capped at 1000, with one extra fetched row used to report truncation. The database grant remains the privilege boundary and should be restricted to the columns or tables this service needs.

Tool inputs use Pydantic models with extra fields forbidden and explicit bounds where relevant. Response formatting is JSON for machine use or compact Markdown for readable summaries. Tests inject HTTP transports and database runners rather than reaching production services.

7. Development checks

```
cd integrations/mcp
python -m pip install -r requirements.txt
python -m pytest -q
python evals/check_answers.py
python -m solarinet_mcp --check
```

The package requires Python 3.10+ and FastMCP SDK 1.x (`mcp>=1.9.4,<2`); the lower bound is needed for the streamable HTTP transport contract used here. At this checkpoint, the suite contains 86 tests and the evaluation has 10 fixture-backed questions.